



Universidade do Minho
Escola de Ciências

Programação Concorrente

Trabalho Prático - G07

Trabalho realizado por: Vasco Leite; Afonso Leal; Gustavo Faria; José
Miranda

Universidade do Minho, Braga, Portugal

[Resumo](#)

[Servidor](#)

[Server](#)

[Account manager](#)

[Score_manager](#)

[Matchmaker](#)

[Tcp_listener](#)

[Game_room](#)

[Player connection](#)

[Cliente](#)

[GameApp](#)

[GameClient](#)

Resumo

Servidor

Server

O módulo `server.erl` é o ponto inicial da aplicação do lado do servidor. Tem como função iniciar o arranque de todos os processos independentes que são necessários para o funcionamento do jogo, nomeadamente o gestor de contas (`account_manager`), o gestor de classificações (`score_manager`), o “matchmaker” e por fim o processo que aceita as ligações de rede (`tcp_listener`).

Account_manager

O “`account_manager.erl`” é o componente encarregue de autenticar os jogadores que tentam acessar o jogo. Ele processa as mensagens de rede enviadas pelas threads de ligação dos clientes, e valida se os credenciais fornecidos num pedido de Login/Cancelamento são corretos ou cria uma nova entrada no sistema no caso de um pedido de Registo.

Estes pedidos de login, cancelamento e registo são síncronos. Esta escolha foi feita para, por exemplo, garantir que um jogador após fazer um pedido de login não continue o seu processo sem primeiro receber a mensagem que autentica ou não os seus credenciais.

Score_manager

O módulo “`score_manager.erl`” atua como o serviço que conta as classificações dos diferentes jogadores (Leaderboard). A sua função é registar

os vencedores no final de cada partida, manter um histórico ordenado das melhores pontuações (número de capturas) desde que o servidor foi iniciado, e fornecer esta lista ao sistema de matchmaking para ser exibida aos jogadores que aguardam no Lobby.

Matchmaker

O `matchmaker.erl` é o coordenador de partidas. Gere a fila de espera dos jogadores autenticados e, se o número de jogadores for suficiente, agrupa-os em salas de jogo (`game_room`). Adicionalmente, envia periodicamente a tabela de classificações para os clientes enquanto estes aguardam na fila.

Tcp_listener

O módulo `"tcp_listener.erl"` é responsável pela comunicação em rede do servidor. A sua única responsabilidade é instanciar uma socket de escuta passiva numa porta especificada (por padrão, a porta 8080) e executar um ciclo contínuo e infinito de aceitação de conexões (`accept_loop`). Sempre que um novo cliente Java estabelece uma ligação à rede, este módulo aceita o descritor de ficheiro da conexão e delega imediatamente o seu controlo a um novo processo dedicado do módulo `"player_connection"`, libertando-se de imediato para aceitar o próximo utilizador.

Game_room

O módulo `"game_room.erl"` atua como o "motor de jogo" (Game Engine) do servidor. É instanciado de forma independente pelo matchmaker sempre que uma nova partida começa. A sua responsabilidade é centralizar toda a simulação: atualizar as posições dos jogadores através de um motor de física, calcular colisões (jogadores vs. objetos e jogadores vs. jogadores), gerir a geração dinâmica do mapa, contabilizar as pontuações e transmitir periodicamente o estado atualizado a todos os clientes ligados.

Player_connection

O módulo “player_connection.erl” atua como a entidade que traduz as comunicações entre um cliente Java específico e os serviços internos do servidor Erlang. Ele encapsula o ciclo de vida completo da socket de um jogador, lendo os pacotes binários brutos da rede, decodificando os comandos pretendidos (Registo, Autenticação, Inputs de Teclado) e reencaminhando-os para os respetivos gestores. Em sentido inverso, escuta as mensagens assíncronas enviadas pela game_room e escreve-as diretamente na socket do cliente para atualizar a interface gráfica em Java.

Cliente

A componente cliente da aplicação foi desenvolvida em Java e encontra-se estruturada de forma a separar a interface gráfica da lógica de comunicação de rede, garantindo que o processamento contínuo das mensagens do servidor não bloqueia a renderização do jogo.

GameApp

A classe GameApp.java atua como o motor gráfico do cliente, recorrendo à biblioteca Processing. A sua responsabilidade principal é gerir a interface visual, desenhando os diferentes estados do jogo a uma taxa de atualização fixa. Quando o jogador aguarda na fila de espera (Lobby), este módulo processa e exibe a tabela de classificações atualizada. Quando a partida inicia, passa a focar-se na renderização da arena, desenhando os vários jogadores e os objetos consumíveis no ecrã com base nas suas coordenadas, ângulos e massas. Adicionalmente, esta classe captura continuamente os inputs do utilizador através do teclado e notifica o cliente de rede das ações. Para evitar condições de corrida e garantir a consistência dos dados, o acesso ao estado partilhado do jogo (jogadores e objetos) é estritamente controlado através de um mecanismo de exclusão mútua otimizado, nomeadamente um trinco de leitura e escrita (ReentrantReadWriteLock). Esta abordagem permite múltiplas leituras simultâneas para a renderização gráfica, mas garante acesso exclusivo durante a escrita dos pacotes de rede. Adicionalmente, as funções de gestão de

estado e comunicação entre threads utilizam métodos nativamente sincronizados (*synchronized*), assegurando a consistência total da aplicação

GameClient

O módulo `GameClient.java` é o responsável por gerir a comunicação TCP bidirecional com o servidor Erlang. Após estabelecer a ligação com o sistema (através da porta 8080), este módulo lança uma *thread* independente (`listenFromServer`) dedicada exclusivamente a escutar pacotes de forma assíncrona. Esta *thread* lê continuamente os pacotes binários que chegam ao descritor de ficheiro do lado do cliente, decodifica o identificador do pacote (como, por exemplo, respostas de autenticação, inícios de partida, atualizações periódicas de posições de todos os intervenientes e o anúncio de fim do jogo) e sinaliza a classe principal (`GameApp`) para atualizar as suas estruturas de dados locais. Em sentido inverso, também encapsula os comandos locais gerados na interface (pedidos de autenticação e alterações nos inputs de movimento), transformando-os em fluxos de bytes e transmitindo-os para o `player_connection` associado no servidor Erlang.